



Módulo 2

Arquitectura Modular

Curso: Terraform Hands-On (GCP)

2 h (120 min)

Objetivos de Aprendizaje

- Diseñar la estructura de un módulo Terraform siguiendo buenas prácticas (`variables.tf` , `outputs.tf` , `versions.tf` , `README.md` , ejemplos).
- Parametrizar correctamente entradas y exponer salidas tipadas y documentadas.
- Consumir módulos desde el **Terraform Public Registry** y desde un **Private Registry** basado en GCS.
- Aplicar **versionado semántico** (`v1.0.0`) y elegir la estrategia de pinning (`=` , `~>` , sin restricción).
- Versionar, empaquetar y publicar un módulo propio en GCS siguiendo el patrón `modules/<name>/<version>/<module>.zip` .
- Consumir el módulo **Cloud SQL** desde la raíz del proyecto AppLocker reutilizando el state remoto del M1.

0. Apertura y objetivos

0.1 Recap del M1

- State remoto en **GCS** con versioning y locking.
- `terraform import` para adoptar infra existente sin drift.
- Workspaces `default` / `dev` / `prod`.
- Caso AppLocker hasta hoy: bucket de state importado, **ningún recurso de aplicación todavía**.

0.2 Pregunta disparadora

Si mañana tenemos 3 entornos (dev / staging / prod) y 2 regiones, ¿copiamos y pegamos el código del Cloud SQL?

- Copiar y pegar el mismo bloque HCL N veces → **anti-patrón clásico**.
- Cuando el recurso se repite, hay una **abstracción pidiendo nacer**: el módulo.
- Hoy aprenderemos a construirla bien: estructura canónica, versionado y registry privado.

0.3 Distribución del tiempo

Fase	Min	%
Apertura y objetivos	5	4%
Bloque teórico	35	29%
Lab guiado	75	63%
Ejercicio autónomo	(fuera)	—
Cierre, Q&A	5	4%

1. ¿Por qué módulos?

Motivación y criterios de extracción

1.1 El anti-patrón: monolito Terraform

- Todo en un único `main.tf`: **imposible de reutilizar**.
- Un único `state` compartido: **reaplicar un recurso fuerza reaplicar todos**.
- Cualquier cambio toca toda la infra → **revisiones gigantes**, miedo a tocar nada.

Regla de oro: si el mismo bloque HCL se repite más de una vez, es candidato a módulo.

1.2 Cuándo extraer un módulo

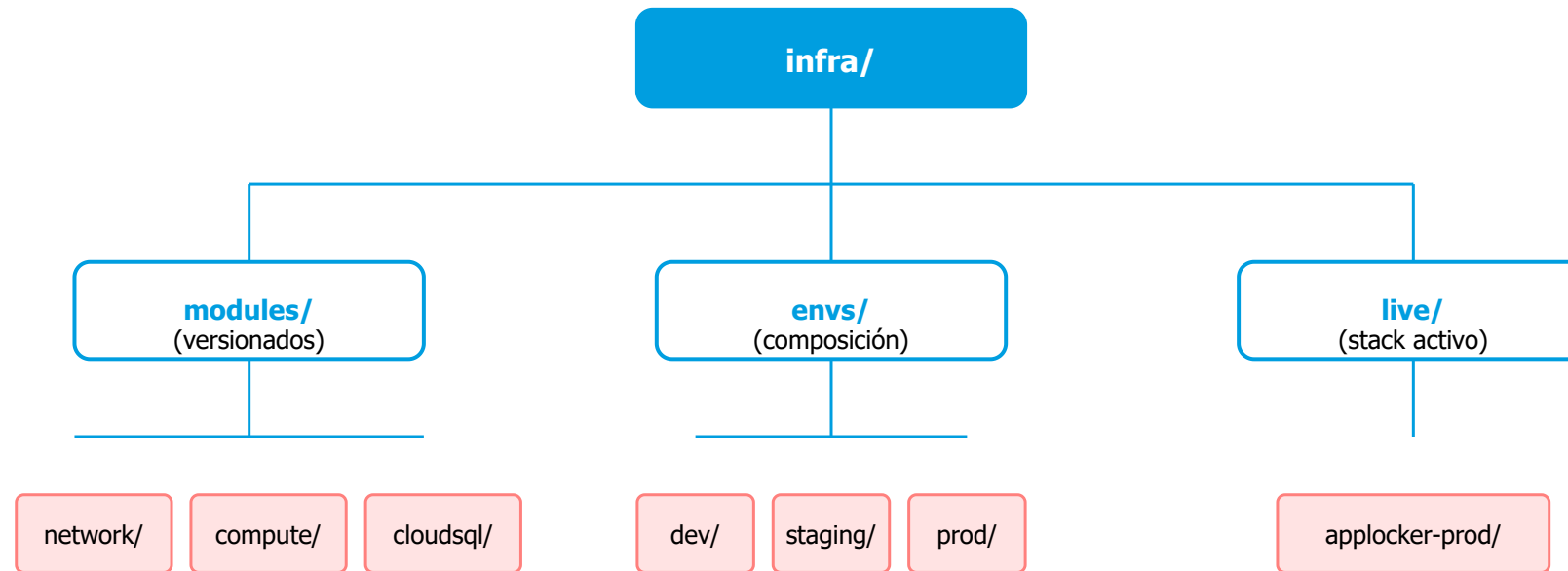
- El recurso se va a crear **más de una vez** (multi-env, multi-region).
- Hay **lógica de negocio** que debe quedar encapsulada (ej. HA + SSL + PITR para Cloud SQL).
- El equipo necesita **contratos estables** entre quien crea infraestructura y quien la consume.

Tip: el módulo Cloud SQL de AppLocker cumple las tres — se reutiliza en dev/staging/prod, encapsula HA y SSL, y libera al consumidor de saber los detalles.

1.3 Cuándo NO extraer

- Recurso **único** que nunca se va a reutilizar.
- **Over-engineering**: módulos de una sola `resource` sin parámetros.
- Bloques que cambian **en su totalidad** entre consumidores (no hay contrato común).

1.4 Jerarquía recomendada para AppLocker



Convención AppLocker

modules/ → bloques reutilizables con SemVer
envs/ → composición por entorno (consume módulos)
live/ → stack activo que se aplica con terraform apply

✦ **modules/** contiene los módulos versionados; **envs/** compone por entorno; **live/** ejecuta el stack activo.

1.5 Mini-lab: detectar duplicación en el M1 (5 min)

1. Abrir el repositorio del M1 y localizar el bloque `google_storage_bucket` ya importado.
2. Imaginar por escrito cómo se vería `main.tf` si tuviéramos 3 buckets idénticos para `dev` / `staging` / `prod`.
3. Identificar qué partes variarían (nombre, labels) y qué partes serían constantes (versionado, UBLA, location).
4. Conclusión: **el bloque entero es candidato a módulo desde ya**, aunque el curso lo enfoca en Cloud SQL.

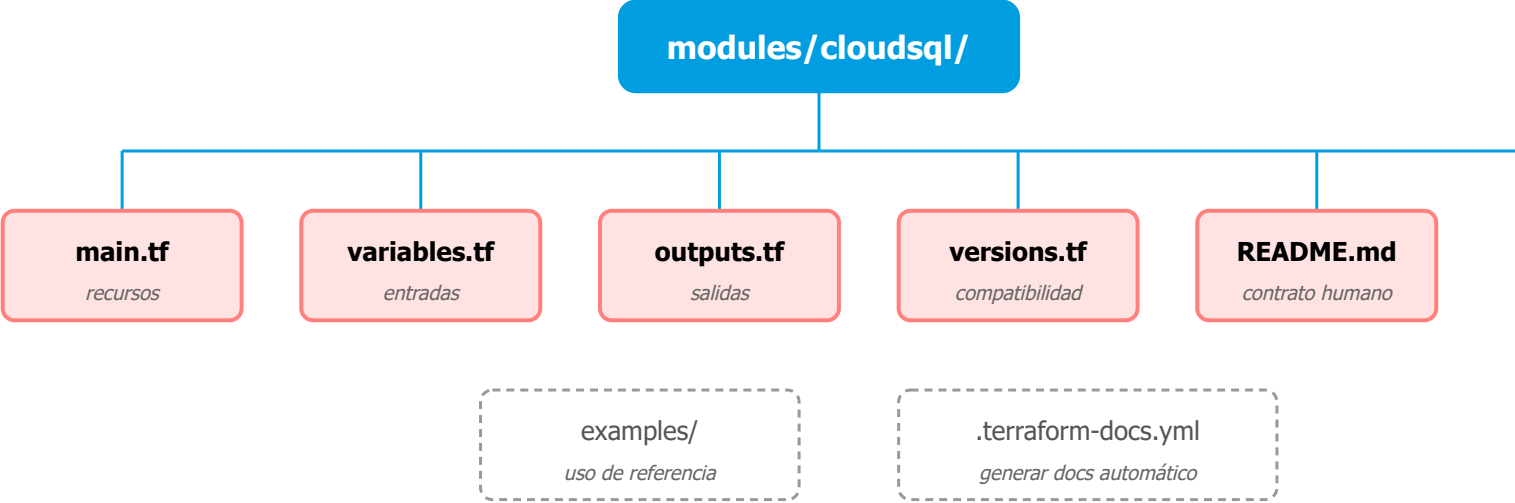
 [Ref. oficial — Terraform Modules](#)

2. Anatomía de un módulo Terraform

Estructura canónica y contrato

2.1 Estructura canónica

Anatomía de un módulo Terraform



Roles por archivo

- — declara los recursos (google_sql_database_instance, etc.)
- — entradas tipadas + validation + description
- — expone al consumidor (sensitive cuando aplique)
- — required_version + required_providers
- — inputs/outputs como tabla + example usage
- y son opcionales

Cada archivo cumple un rol: **main** declara, **variables** recibe, **outputs** expone, **versions** fija compatibilidad.

2.2 `versions.tf` — la pieza olvidada

```
terraform {
  required_version = "~> 1.5"

  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}
```

Anti-patrón: dejar el provider sin `version`. Tu build deja de ser reproducible el día que el provider publique una breaking change.

2.3 `variables.tf` — entradas tipadas

```
variable "project_id" {
  type      = string
  description = "ID del proyecto GCP donde se desplegará la instancia."
  nullable  = false
}

variable "tier" {
  type      = string
  description = "Tier de la instancia (ej. db-custom-2-7680)."
  default    = "db-custom-2-7680"

  validation {
    condition     = can(regex("^db-[a-z0-9-]+$", var.tier))
    error_message = "tier debe tener formato db-..."
  }
}
```

📌 `description` es **obligatorio** en módulos publicados (Terraform 1.6+).

2.4 `outputs.tf` — exponer lo justo

```
output "connection_name" {
  value      = google_sql_database_instance.this.connection_name
  description = "Connection name de Cloud SQL (project:region:instance)."
```

```
}

output "private_ip" {
  value      = google_sql_database_instance.this.private_ip
  description = "IP privada de la instancia (vacía si es pública)."
```

```
}

output "db_password" {
  value      = random_password.db.result
  sensitive  = true
}
```

Regla: exponer solo lo que el consumidor necesita. Marcar como `sensitive` todo lo que contenga credenciales.

2.5 Convención de nombres

- Para módulos propios del equipo: `terraform-<provider>-<recurso>` (ej. `terraform-google-cloudsql`).
- Esta convención **facilita el discovery** en el registry interno.
- Para módulos públicos HashiCorp usa el namespace `hashicorp/<provider>-<name>` .

2.6 Lab: esqueleto del módulo `cloudsql` (15 min)

1. Crear la estructura de directorios:

```
mkdir -p infra/modules/cloudsql/examples/simple
```

2. `versions.tf` con `required_version = "~> 1.5"` y provider `hashicorp/google` con constraint.

3. `variables.tf` con las entradas mínimas (`project_id` , `name` , `region` , `tier` , `availability_type` , `database_version` , `disk_size` , `private_network` , `deletion_protection`).

4. `main.tf` moviendo el bloque `google_sql_database_instance` del M3 (se adelanta para que M3 herede el módulo).

5. `outputs.tf` con `connection_name` , `self_link` , `private_ip` , `database_name` , `user_name` .

6. Verificar con `terraform fmt` y `terraform validate` desde dentro del módulo.



[Ref. oficial — Module development](#)

3. Versionado y consumo de módulos

SemVer, constraints y source types

3.1 Source types que Terraform soporta

- **Local:** `source = "./modules/cloudsql"`
- **Git (SSH):** `source = "git@github.com:acme/tf-modules.git//cloudsql?ref=v1.0.0"`
- **Git (HTTPS):** igual sin SSH.
- **Terraform Public Registry:** `source = "terraform-google-modules/sql/google"` (sin prefix `hashicorp/`).
- **GCS Private Registry:** `source = "gcs::https://www.googleapis.com/storage/v1/<bucket>/modules/cloudsql.zip"`

3.2 Versionado SemVer

- **MAJOR** (breaking): cambio incompatible en variables u outputs.
- **MINOR** (feature): nueva variable opcional, nuevo output.
- **PATCH** (fix): bugfix sin cambio de contrato.

Ojo: Terraform **no acepta** `v1` ni `1` como SemVer válido. Siempre `v1.0.0`.

 [Ref. oficial — semver.org](https://semver.org)

3.3 Constraints de versión

Constraint	Acepta	Uso
"1.2.0"	Solo 1.2.0	Pin exacto (auditoría)
"~> 1.2"	>= 1.2, < 2.0	Prod conservador
"~> 1.2.0"	>= 1.2.0, < 1.3.0	Bugfixes solo
">= 1.2.0"	Sin tope superior	Peligroso en prod

Regla práctica: en producción, ~> con MINOR fijo. La MAJOR la subes cuando rompes contrato.

3.4 Anti-patrones de versionado

- Apuntar a `main` / `master` : **no reproducible**.
- Versionar sin changelog: el equipo no sabe qué cambió entre `v1.4.2` y `v1.5.0`.
- Mismo tag para múltiples providers / recursos (rompe el discovery).
- Tag Git `v1` sin parche — Terraform lo rechaza.

3.5 Lab: consumir desde el Public Registry (10 min)

1. Cambiar a `envs/dev/main.tf` del proyecto AppLocker.
2. Comentar (temporalmente) el recurso local `google_sql_database_instance`.
3. Sustituir por el módulo público:

```
module "cloudsql" {  
  source = "terraform-google-modules/sql/google"  
  version = "~> 20.0"  
  
  project_id = var.project_id  
  region     = "us-central1"  
  name       = "applocker-db-dev"  
  database_version = "POSTGRES_15"  
  tier       = "db-f1-micro"  
  
  ip_configuration = {  
    ipv4_enabled = false  
    private_network = module.network.vpc_self_link  
  }  
}
```

4. `terraform init` y verificar la descarga del módulo.
5. `terraform validate` y `terraform plan -no-color` para confirmar no drift.



[Ref. oficial — Module sources](#)

4. Private Module Registry con GCS

Publicar y consumir módulos internos

4.1 ¿Por qué un registry privado?

- Módulos internos con **lógica de negocio** que no pueden ser públicos.
- Control de acceso granular (solo el equipo de plataforma escribe; todos leen).
- **Latencia baja:** el bucket está en la misma región que el cluster.
- Reutilización del bucket de state del M1 con prefijo `modules/` separado por IAM.

4.2 GCS como registry

- Terraform lo soporta nativamente desde **1.1+**.
- Patrón de path obligatorio: `gs://<bucket>/modules/<name>/<version>/<module>.zip`.
- El zip debe contener **un solo directorio raíz** con los `.tf`.
- Cada versión = un objeto nuevo en GCS.

Crítico: el path **debe** seguir ese patrón exacto. Cualquier variante rompe el discovery.

4.3 IAM mínimo

Rol	Quién	Por qué
<code>roles/storage.objectAdmin</code>	Pipeline CI/CD	Publicar módulos
<code>roles/storage.objectViewer</code>	Desarrolladores	Descargar en <code>terraform init</code>

Tip: el formador debe mostrar la consola IAM con las dos cuentas separadas.

 [Ref. oficial — Module Registry Protocol](#)

4.4 Lab: publicar y consumir el módulo privado (15 min)

1. Empaquetar el módulo:

```
cd infra/modules/cloudsql
zip -r /tmp/cloudsql.zip . -x "*.tfstate*" -x ".terraform/*"
```

2. Subir al bucket del M1 con el path canónico:

```
gsutil cp /tmp/cloudsql.zip \
gs://applocker-tf-state-<sufijo>/modules/cloudsql/1.0.0/cloudsql.zip
```

3. Crear el tag Git `v1.0.0`:

```
git tag -a v1.0.0 -m "Initial release: Cloud SQL module for AppLocker"
git push origin v1.0.0
```

4. Cambiar el source en `envs/dev/main.tf`:

```
module "cloudsql" {
  source = "gcs::https://www.googleapis.com/storage/v1/applocker-tf-state-<sufijo>/modules/cloudsql/1.0.0/cloudsql.zip"
  version = "1.0.0"
  # ... mismas variables
}
```

5. `terraform init -upgrade` para forzar la descarga desde GCS.

6. `terraform validate` y `terraform plan`: el plan debe ser idéntico al del lab 3.5.

5. Ejercicio autónomo (fuera del horario)

5.1 Modalidad B — decisión de diseño

- Tiempo estimado: **25 min.**
- Modalidad: **decisión de diseño con trampa** sobre SemVer.
- Entregable: `module-2/exercises/changelog-m2-ej2.md` .
- Trampa: tres cambios simultáneos en un solo PR, dos son breaking.

Ver enunciado completo en: `module-2/exercises/exercise-2.md` .

6. Cierre, Q&A y preparación de M3

6.1 Recap del módulo

- ¿Por qué módulos?: regla del "más de una vez" + lógica encapsulada.
- Anatomía canónica: `main`, `variables`, `outputs`, `versions`, `README`, `examples`.
- SemVer + constraints (`~>` en prod) + tag Git = zip versionado.
- Private Registry en GCS: `modules/<name>/<version>/<module>.zip`.

6.2 Errores comunes que el formador ha visto

- Olvidar `versions.tf` y romper la reproducibilidad del build.
- Versionar `v1` en lugar de `v1.0.0` (Terraform no lo acepta).
- Apuntar el source a `main` y que el lunes no compile.
- Cambiar el tipo de una variable sin subir MAJOR → plan destruye todos los recursos.

6.3 Lo que el alumno debe tener al terminar

- Esqueleto del módulo `infra/modules/cloudsql/` con sus 5 archivos canónicos.
- Zip `v1.0.0` publicado en `gs://<bucket>/modules/cloudsql/1.0.0/cloudsql.zip`.
- Tag Git `v1.0.0` en el repo de módulos.
- `envs/dev/main.tf` consumiendo el módulo desde GCS con `version = "1.0.0"`.

6.4 Pregunta abierta

¿Qué otro recurso de AppLocker será candidato a módulo en M3?

6.5 Preparación de M3

- Repasar `gcloud compute` (cubierto en `anexo/gc-in-a-nutshell.md`).
- Confirmar que el proyecto GCP tiene **billing activo** y las APIs necesarias habilitadas.
- Limpiar el bucket de estado de pruebas de M1 si quedó drift.
- Realizar el ejercicio autónomo (25 min) antes de la sesión de M3.

Resumen

Puntos Clave

- Un **módulo Terraform** es una abstracción reutilizable: extraer cuando se repite + encapsula lógica + define contrato.
- Estructura canónica: `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`, `examples/`.
- `versions.tf` **es obligatorio**: sin él, el build deja de ser reproducible.
- **SemVer** + constraints `~>` en prod: `MAJOR` cuando rompes contrato, `MINOR` cuando añades, `PATCH` cuando arreglas.
- **Private Registry en GCS**: path canónico `modules/<name>/<version>/<module>.zip`, IAM separado por rol.
- Anti-patrones: `v1` sin parche, source a `main`, cambiar tipo de variable sin MAJOR.

Referencias

Recursos oficiales

- [Terraform Modules — Documentación oficial](#)
- [Module development — Documentación oficial](#)
- [Module sources — Documentación oficial](#)
- [Semantic Versioning — semver.org](#)
- [Terraform Public Registry](#)
- [GCP Provider — Documentación oficial](#)
- [Módulo público de referencia — `terraform-google-modules/sql/google`](#)
- [terraform-docs — generador de docs de módulos](#)
- [GCP + Terraform — Guía oficial](#)

📌 Anexo interno del curso: `anexo/gc-in-a-nutshell.md` (secciones 2.2, 2.4 y 4).



¡Gracias!

Módulo 2 Completado

Siguiente: Módulo 3 — Compute Engine, Instance Templates y MIG